Week 10: ELT Deepdive (dbt)

Keeyong Han

Table Of Contents

1. Recap of Midterm Part A & Week 8
2. Best Practices in Data Pipelines
3. dbt Overview
4. dbt Installation & Configuration
5. dbt Models - input
6. dbt Models - output
7. Break
8. dbt snapshot
9. dbt test
10. dbt docs
11. Integration of dbt + Airflow
12. Lab #2
13. Homework #7

Midterm Review

Questions to Review (Part A)

- SQL
  - Which function replaces NULL values in SQL with specified values? COALESCE
  - Which SQL command is used in SNOWFLAKE to merge existing data and insert new records? MERGE not UPSERT
- Airflow
  - Mainly for batch data processing
  - An open source project started at Airbnb
  - “45 * * * *”: DAG running every hour at 45th minute

Recap of Week 8

The Simplest ELT Implementation

- GitHub Code

```
CREATE OR REPLACE TABLE dev.analytics.session_summary AS
SELECT u.*, s.ts
FROM dev.raw_data.user_session_channel u
JOIN dev.raw_data.session_timestamp s ON u.sessionId=s.sessionId;
```

CTAS

```
SELECT sessionId, COUNT(1) AS cnt
FROM dev.analytics.session
GROUP BY 1
ORDER BY 2 DESC
LIMIT 1
```

Primary Key Uniqueness Check
cnt should be 1

Increasing Importance of Data Quality

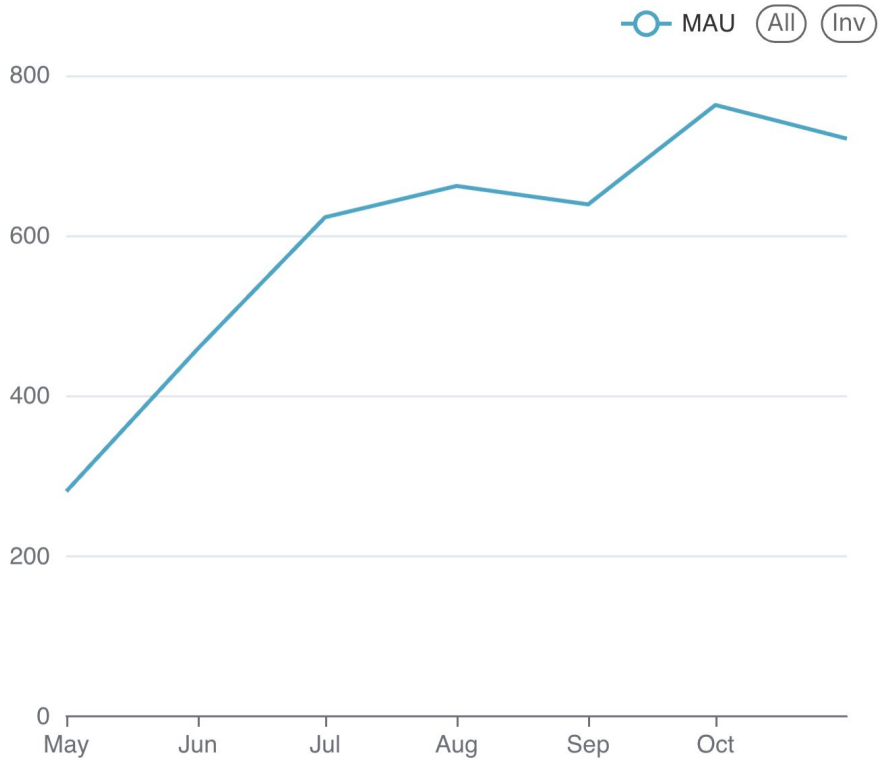
- Input/output checks (in particular for ELTs)
- More diverse quality inspections
- Lineage checks
- Understanding data change history

This is why **dbt** is needed

- Maintaining data quality is more important than ever
  - Reducing cost/effort and increasing productivity
  - Main reasons to have **data governance**

Superset or Preset or Tableau

- Draw a MAU chart using analytics.session_summary table



Best Practices in Data Pipelines

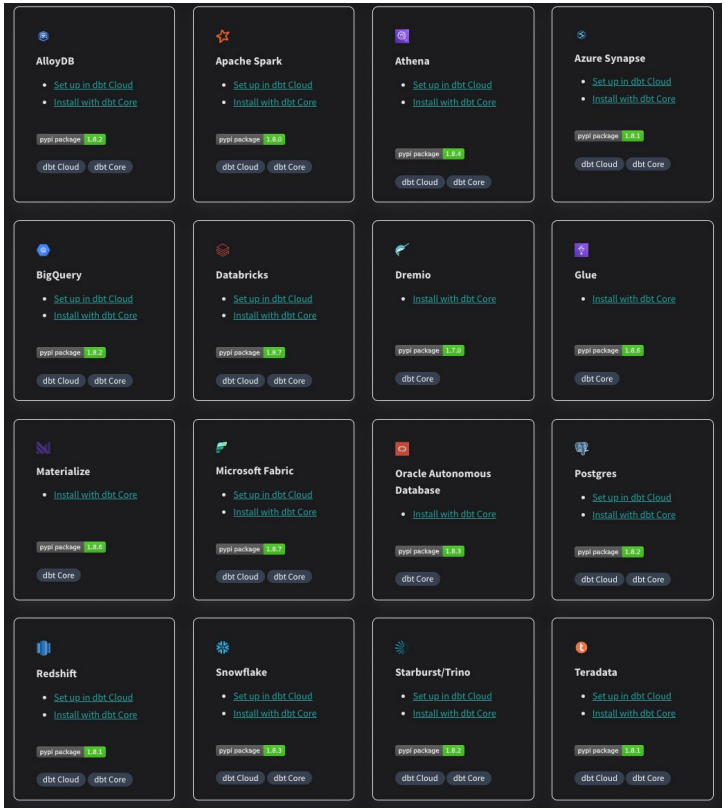
Things to Remember When Writing Data Pipelines

- Collect information about the data pipelines is crucial.
  - Pay attention to business owners and data lineage (tables and columns).
  - A **data catalog** is ultimately needed
  - Source of truths
- Ensure data quality checks
  - Focus on both input and output data.
- Better to fail cleanly rather than attempting sloppy recoveries
- Use Full Refresh whenever possible.
  - If you must use Incremental Updates, make it easy to backfill — this is why Airflow is needed.
- Backfill should be easy
- Regular cleaning (data, tables, data pipelines, …) is essential
- Write a post-mortem report after every data pipeline incident

dbt Overview

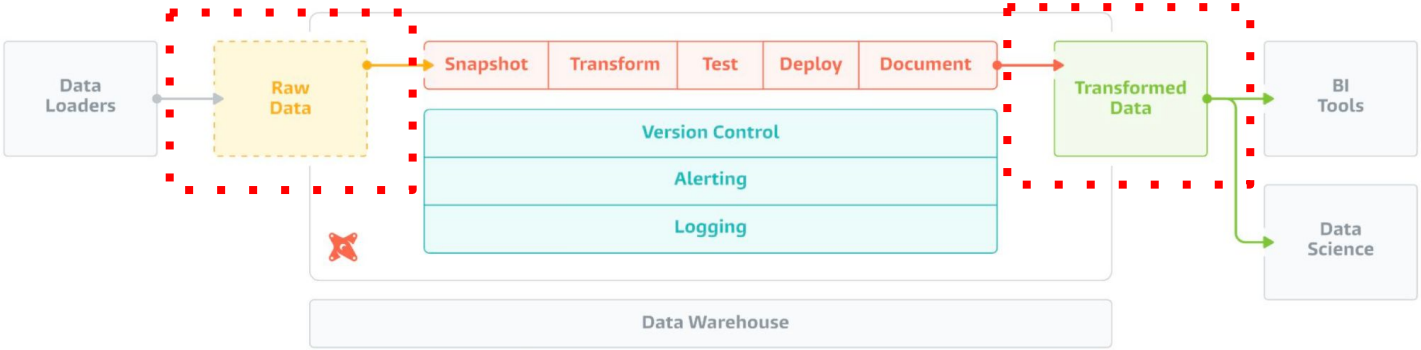
What is dbt?

- Data Build Tool (https://www.getdbt.com/)
  - ELT Open-source: In-warehouse data transformation
  - Commercialized by a company called dbt Labs
    - Valued at $4.2B once
  - Coined the term “Analytics Engineer”
- Supports various data warehouses
  - Redshift, **Snowflake**, BigQuery, Spark
- A cloud version is also available: dbt Cloud



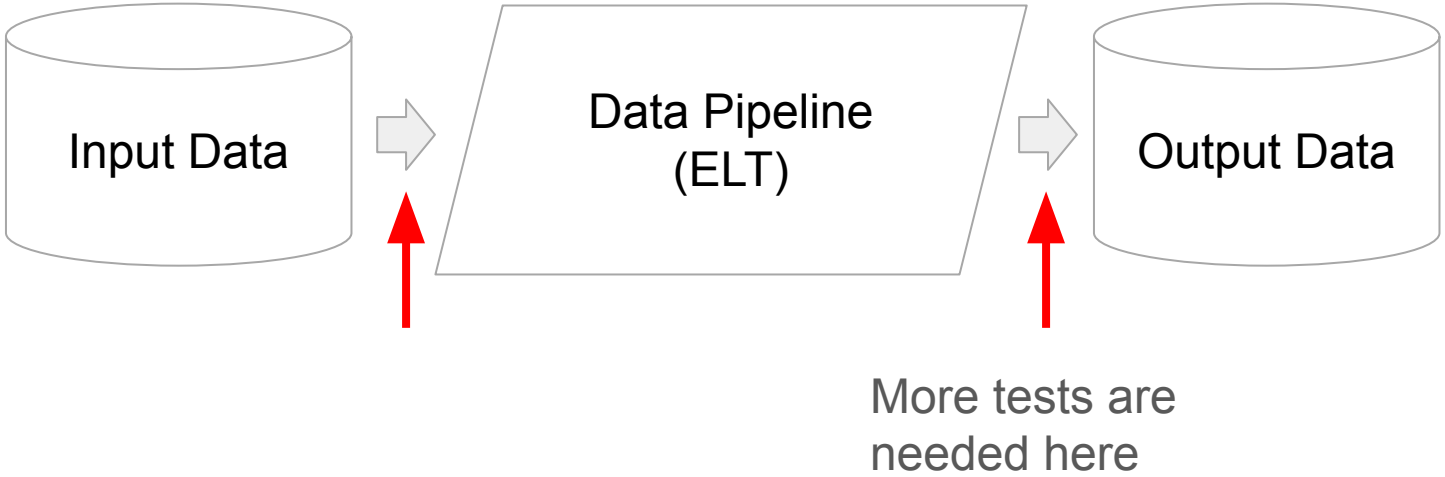
dbt Components

- CTAS (Create Table As Select) on steroids
- Data Models (models):
  - Manages tables in several tiers
  - Lineage tracking
  - Includes Table, View, CTE, etc.
- Data Quality Validation (tests)
- Snapshots: Poor man’s time travel



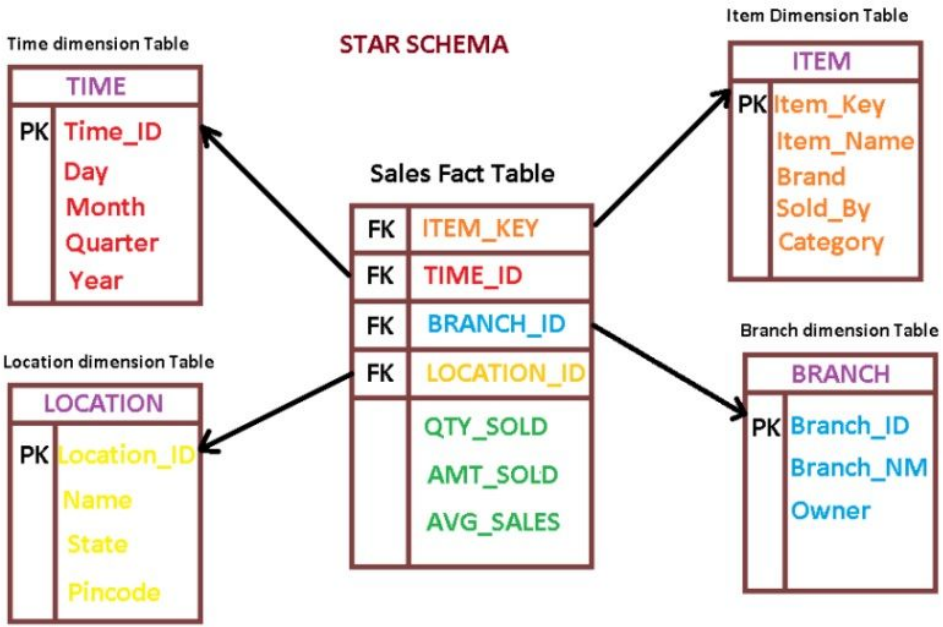
When to Use dbt (1)

- Easy to understand data changes and enable rollbacks if necessary
  - It is best to use along with Git
- Data quality testing and error reporting (“tests”)



When to Use dbt (2)

- Incremental updates for **fact** tables: mostly append only table
- Change tracking for **dimension** tables (“history/archive” tables)



Ideally fact table should be much larger than dimension tables.
Ideally fact table should be immutable (goal of **star** schema)

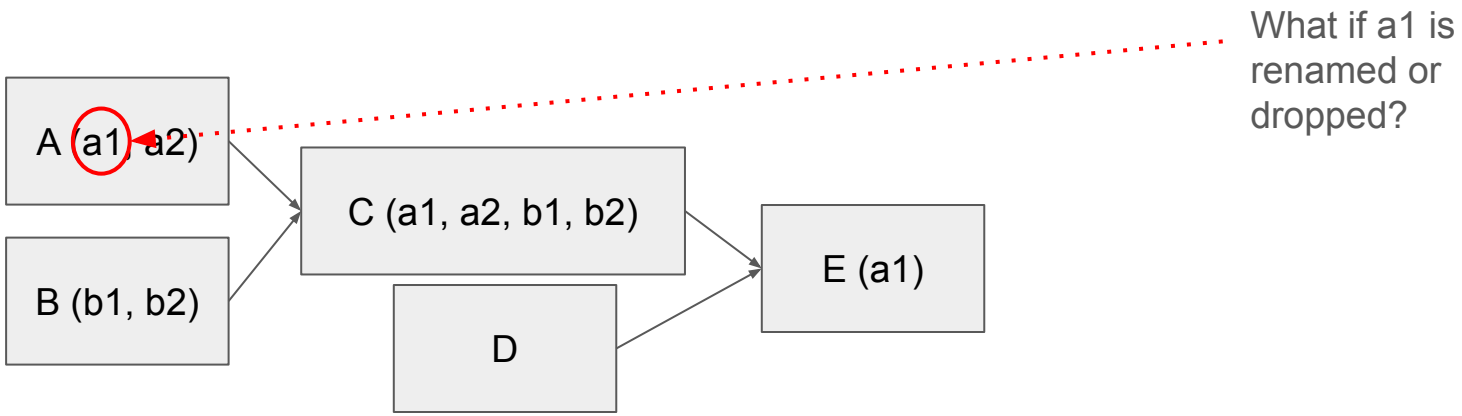
Fact Table and Dimension Table Examples

- Sales Data
  - Fact table: Transaction (numeric facts and contains foreign keys to dimension table)
  - Dimension tables: Product, Customer, …
- Ride Sharing Data
  - Fact able: Trip
  - Dimension tables: Rider, Driver, City, …

- Fact tables are most likely immutable
- Dimension tables can change more frequently
- Alternatives of Fact/Dimension Table: Denormalized Table

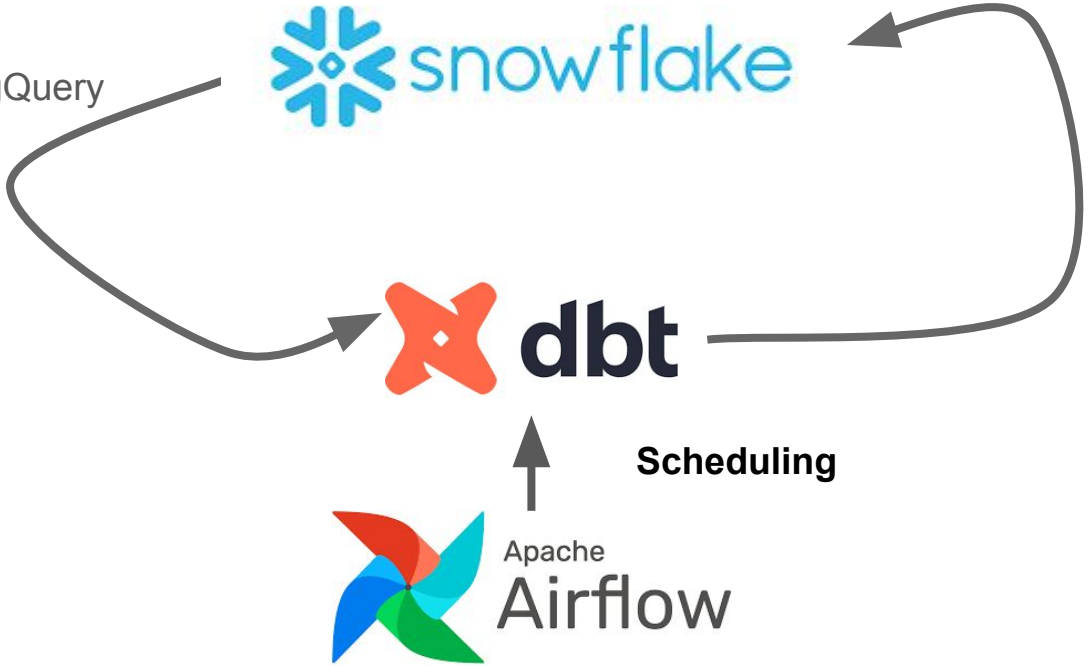
When to Use dbt (3)

- Easy documentation generation
- Ability to track **data lineage**



Tech stack along with dbt

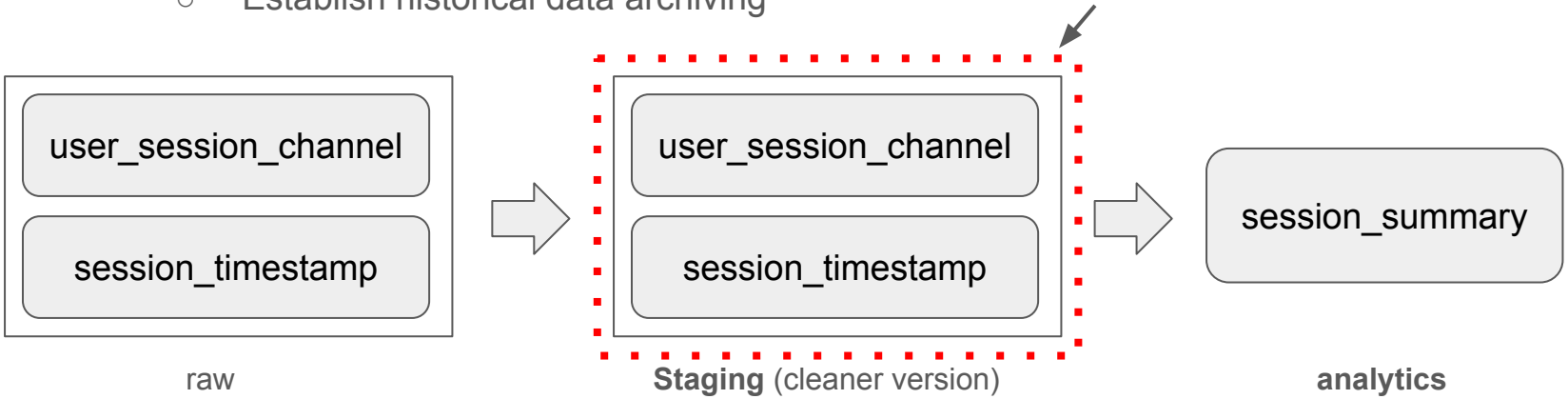
- dbt: CTAS on steroids
- Data Warehouse
  - Redshift/Spark/Snowflake/BigQuery
- Airflow



What are we going to do?

- Migrate analytics.session_summary CTAS DAG to dbt
  - Still use Airflow to schedule the dbt runs
- Leverage dbt capabilities
  - Implement data tests
  - Generate data lineage charts
  - Author comprehensive documentation
  - Establish historical data archiving

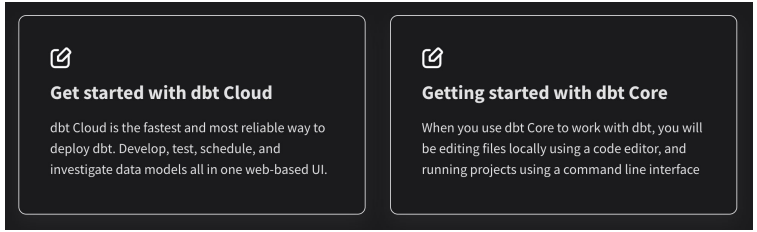
This can be tables or views or CTE depending on your configuration



dbt Installation & configuration

dbt Usage Procedure

- Install dbt
  - dbt Cloud vs. dbt Core
  - Git is commonly used together with
- Configure dbt environment
- Set up Connector
  - Connector is the underlying data system (Snowflake, Redshift, Spark, ...)
- Data Modeling (tiers)
  - Raw Data -> Staging -> Core (analytics)
- Write Test Codes
- (If necessary) Configure Snapshots



dbt Installation: Proceed with local version (1.11.2)

- Run the following command in your terminal
  - This command also installs the dbt-core module

pip3 install dbt-snowflake

If you get an error here, try to set up a Python virtual environment:

python3 -m venv ~/dbt_env
source ~/dbt_env/bin/activate
pip install dbt-snowflake

```
keeyong week10 % pip3 install --upgrade dbt-snowflake
Collecting dbt-snowflake
  Downloading dbt_snowflake-1.8.3-py3-none-any.whl.metadata (3.2 kB)
Collecting dbt-common<2.0,>=1.0.4 (from dbt-snowflake)
  Downloading dbt_common-1.10.0-py3-none-any.whl.metadata (5.3 kB)
Collecting dbt-adapters<2.0,>=1.1.1 (from dbt-snowflake)
  Downloading dbt_adapters-1.7.0-py3-none-any.whl.metadata (2.5 kB)
Collecting snowflake-connector-python~=3.0 (from snowflake-connector-python[secure-local-stora
ge]~=3.0->dbt-snowflake)
  Downloading snowflake_connector_python-3.12.2-cp311-cp311-macosx_11_0_x86_64.whl.metadata (6
5 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 65.3/65.3 kB 1.9 MB/s eta 0:00:00
Collecting dbt-core>=1.8.0 (from dbt-snowflake)
  Downloading dbt_core-1.8.7-py3-none-any.whl.metadata (3.9 kB)
Collecting agate (from dbt-snowflake)
  Downloading agate-1.12.0-py2.py3-none-any.whl.metadata (3.2 kB)
```

Configure dbt environment

Use your Snowflake connection information here

```
% dbt init build_mau
Which database would you like to use?
[1] snowflake

(Don't see the one you want? https://docs.getdbt.com/docs/available-adapters)

Enter a number: 1
account (https://<this_value>.snowflakecomputing.com): SFEDU02-LVB17920
user (dev username): RACCOON
[1] password
[2] keypair
[3] sso
Desired authentication type option (enter a number): 1
password (dev password):
role (dev role): TRAINING_ROLE
warehouse (warehouse name): RACCOON_QUERY_WH
database (default database that dbt will build objects in): USER_DB_RACCOON
schema (default schema that dbt will build objects in): analytics
threads (1 or more) [1]:
06:02:48  Profile build_mau written to /Users/jobox/.dbt/profiles.yml using target's profile_template.yml and your supplied values.
Run 'dbt debug' to validate the connection.
```

What did it create in the project?

Main configuration file in YAML format

% ls -tl build_mau

```
total 16
-rw-r--r--  1 jobox  staff  1274 Oct 14 14:54 dbt_project.yml
drwxr-xr-x  3 jobox  staff    96 Oct 14 14:53 tests
drwxr-xr-x  3 jobox  staff    96 Oct 14 14:53 snapshots
drwxr-xr-x  3 jobox  staff    96 Oct 14 14:53 seeds
drwxr-xr-x  3 jobox  staff    96 Oct 14 14:53 models
drwxr-xr-x  3 jobox  staff    96 Oct 14 14:53 macros
drwxr-xr-x  3 jobox  staff    96 Oct 14 14:53 analyses
-rw-r--r--  1 jobox  staff   571 Oct 14 14:53 README.md
```

dbt_project.yml

name: 'build_mau'
version: '1.0.0'
config-version: 2

profile: 'build_mau'

model-paths: ["models"]
analysis-paths: ["analyses"]
test-paths: ["tests"]
seed-paths: ["seeds"]
macro-paths: ["macros"]
snapshot-paths: ["snapshots"]

The folder where outputs are stored

target-path: "target" # folder to store compiled SQL files
clean-targets: # directories to be removed by `dbt clean`
- "target"
- "dbt_packages"
models:
build_mau:
example:
+materialized: view

Delete these two lines and remove the example folder under models folder

```
keeyong build_mau % cd models
keeyong models % ls -tl
total 0
drwxr-xr-x  5 jobox  staff  160 Oct 14 14:53 example
keeyong models % rm -rf example
keeyong models %
```

yml (or yaml): Popular as a Config File Format

```
# Comments start with a #

# Key-value pairs are separated by a colon and a space
name: John Doe  # { “name”: “John Doe”, “age”: 30 }
age: 30

# Lists are denoted by a hyphen and a space
hobbies:     # { “hobbies”: [ “reading”, “hiking” ]
  - reading
  - hiking

# Nested key-value pairs are indented with two spaces
contact:
  email: john.doe@example.com
  phone:
    home: 555-1234
    work: 555-5678
```

dbt Models - input

What is dbt Models?

- Basic building blocks in creating ELT tables
  - Can exist in the form of tables, views, or CTEs
  - A sequence of data transformations is defined
- Defines raw input, intermediate/staging, and final tables
  - Tiers (raw, staging, core, ...)
  - raw (raw_data) => **staging** => core/final (analytics)

```
total 16
-rw-r--r--  1 jobox  staff  1274 Oct 14 14:54 dbt_project.yml
drwxr-xr-x  3 jobox  staff    96 Oct 14 14:53 tests
drwxr-xr-x  3 jobox  staff    96 Oct 14 14:53 snapshots
drwxr-xr-x  3 jobox  staff    96 Oct 14 14:53 seeds
drwxr-xr-x  3 jobox  staff    96 Oct 14 14:53 models
drwxr-xr-x  3 jobox  staff    96 Oct 14 14:53 macros
drwxr-xr-x  3 jobox  staff    96 Oct 14 14:53 analyses
-rw-r--r--  1 jobox  staff   571 Oct 14 14:53 README.md
```

dbt Models: input and output

- We will create two folders (input and output) under models folder
- Input
  - Defines input (raw) and intermediate (staging, src) data
  - Raw data is in raw_data schema in Snowflake
  - Staging data will be defined as CTE (or Views)
- Output
  - Defines final (core) data under analytics schema
  - Core data is defined as Physical Table
- All of these exist as SQL files under the models folder
  - Basically consists of SELECT statements + some reference variables
  - This reference variables allow for lineage tracking

What is View?

- **Virtual** table created based on SELECT results
  - Provides results from a part of an existing table or joined results from multiple tables
  - CREATE VIEW name AS SELECT ...
- Advantages of Views
  - Data abstraction: Users can directly access needed data through the view. No need to know the original data
  - Simplification of complex queries: Using SQL (View) simplifies complex queries
- Disadvantages of Views
  - May take time as the query is executed every time
  - Execution may fail if changes to the original data are unknown

What is CTE (Common Table Expression)?

```
WITH temp1 AS (
    SELECT k1, k2
    FROM t1
    JOIN t2 ON t1.id = t2.foreign_id
), temp2 AS (
…
)
SELECT *
FROM temp1 t1
JOIN temp2 t2 ON …
```

Example)

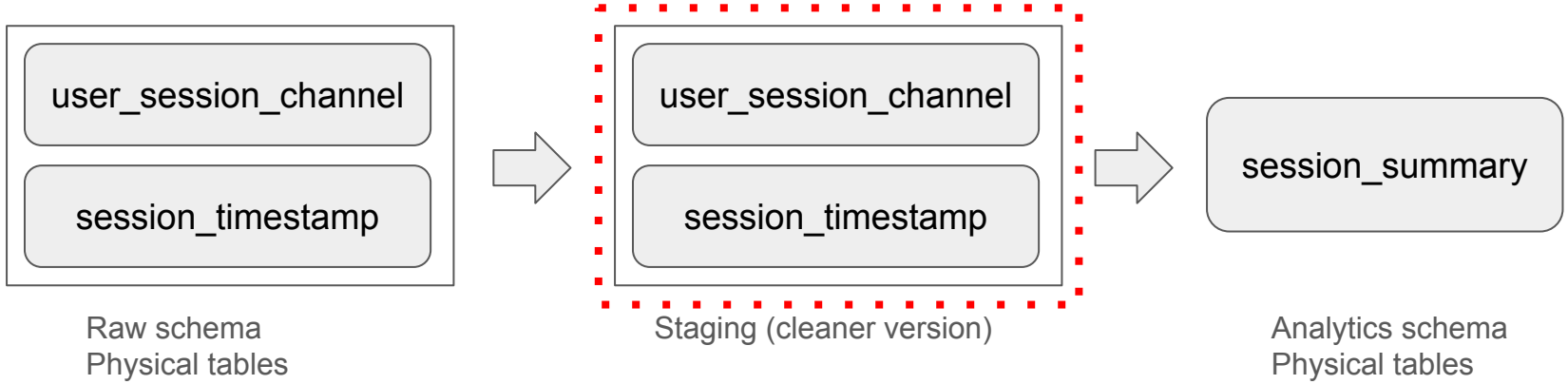
```
WITH src_user_event AS (
    SELECT * FROM raw_data.user_event
)
SELECT
    user_id,
    datestamp,
    item_id,
    clicked,
    purchased,
    paidamount
FROM
    src_user_event
```

These are just SQL statements. No presence in underlying database

dbt Models: input and output

```
keeyong models % mkdir input output
keeyong models % ls -tl
total 0
drwxr-xr-x  2 jobox  staff  64 Oct 14 22:23 output
drwxr-xr-x  2 jobox  staff  64 Oct 14 22:23 input
```

This can be tables or views or **CTE** depending on your configuration



Make sure they exist in your database.raw schema: link

Will be defined in input folder

Will be defined in output folder

models/input - user_session_channel.sql

```
SELECT
    userId,
    sessionId,
    channel
FROM {{ source('raw', 'user_session_channel') }}
WHERE sessionId IS NOT NULL
```

Doing some cleanup

models/input - session_timestamp.sql

```
SELECT
    sessionId,
    ts
FROM {{ source('raw', 'session_timestamp') }}
WHERE sessionId IS NOT NULL
```

Doing some cleanup

models - source.yml (this is to define non dbt inputs)

```yaml
version: 2

sources:
  - name: raw
    database: USER_DB_RACCOON
    schema: raw
    description: "Schema which stores raw tables (ETL)"
    tables:
      - name: user_session_channel
        identifier: user_session_channel
        description: "Session data including userId, sessionId, and channel"
      - name: session_timestamp
        identifier: session_timestamp
        description: "Session data including timestamp"
  - name: analytics
    database: USER_DB_RACCOON
    schema: analytics
    description: "Schema with stores analytics tables (ELT)"
    tables:
      - name: session_summary
        identifier: session_summary
        description: "Complete session info"
```

{{ source('**raw**', '**user_session_channel**') }} refers to **database.schema.identifier**

Replace database with yours!

Sometime changing underlying raw tables is required. Finding and updating all of them manually can be time consuming. That's why source notation is needed.

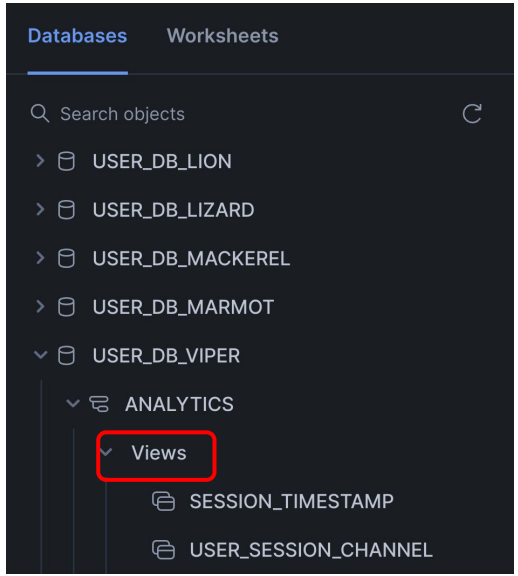
dbt run

- Make sure you run this command in **the root folder** of your dbt project

```
keeyong build_mau % dbt run
06:27:03  Running with dbt=1.9.3
06:27:04  Registered adapter: snowflake=1.9.2
06:27:05  Found 2 models, 3 sources, 474 macros
06:27:05
06:27:05  Concurrency: 1 threads (target='dev')
06:27:05
06:27:05  1 of 2 START sql view model analytics.session_timestamp ........................ [RUN]
06:27:06  1 of 2 OK created sql view model analytics.session_timestamp ................... [SUCCESS 1
.42s]
06:27:06  2 of 2 START sql view model analytics.user_session_channel ..................... [RUN]
06:27:06  2 of 2 OK created sql view model analytics.user_session_channel ................ [SUCCESS 1
.35s]
06:27:06
06:27:06  Finished running 2 view models in 0 hours 0 minutes and 1.58 seconds (1.58s).
06:27:06
06:27:06  Completed successfully
```

Let’s check two tables as staging under analytics schema

- “dbt run” executes various SQL statements defined in the project
- Two staging tables are views under analytics schema
  - What if I want to change it to CTE



What is Materialization?

- It refers to creating new data (tables) from input data (tables).
  a. Usually, additional transformations or data cleanups are performed at this stage.
  b. It can be done at the file or project level.
- You can also run dbt run with various parameters.
- 4 types of materialization in dbt
  a. **View**: Use when you need the latest data and have low performance requirements.
  b. **Table**: Use when you need fast queries and can afford to store physical data.
  c. **Incremental**: Use when working with large datasets that are frequently updated, but where only small portions of the data change at a time. It will be a table
  d. **Ephemeral (CTE)**: Use for lightweight, temporary transformations during a dbt run **without** creating physical objects (SQL level transformation)

Let’s change two staging tables from View to CTE

- Default setting is to build everything as **views**
- Edit dbt_project.yml

```
models:
  build_mau:
# example:
#   +materialized: view
```

Build every data under example folder as view

By default the project will materialize everything as views

```
models:
  build_mau:
  +materialized: view
  output:
    +materialized: table
  input:
    +materialized: ephemeral
```

Data under input folder should be CTEs while those under output folder should be tables

dbt run

```sql
DROP VIEW analytics.user_session_channel;
DROP VIEW analytics.session_timestamp;
```

- Now it doesn’t show anything about view: dbt compile vs. dbt run
- staging tables are embedded and built as CTEs (so no actual materialization)

```
keeyong build_mau % dbt run
06:33:30  Running with dbt=1.9.3
06:33:31  Registered adapter: snowflake=1.9.2
06:33:31  Unable to do partial parsing because a project config has changed
06:33:32  [WARNING]: Configuration paths exist in your dbt_project.yml file which do not
urces.
There are 1 unused configuration paths:
- models.build_mau.output
06:33:32  Found 2 models, 3 sources, 474 macros
06:33:32
06:33:32  Concurrency: 1 threads (target='dev')
06:33:32
06:33:32
06:33:32  Finished running  in 0 hours 0 minutes and 0.01 seconds (0.01s).
```

dbt Models - output

models/output - session_summary.sql

```
SELECT u.userId, u.sessionId, u.channel, st.ts
FROM {{ ref("user_session_channel") }} u
JOIN {{ ref("session_timestamp") }} st ON u.sessionId = st.sessionId
```

- Never hardcode a table name!

- source notion is to refer to first level input tables.
- ref notation is to refer to dbt data models.
- Source and ref notation will make **lineage** creation possible.

dbt run

- Make sure you run this command in the root folder of your dbt project

```
keeyong build_mau % dbt run
06:36:10  Running with dbt=1.9.3
06:36:11  Registered adapter: snowflake=1.9.2
06:36:11  Found 3 models, 3 sources, 474 macros
06:36:11
06:36:11  Concurrency: 1 threads (target='dev')
06:36:11
06:36:12  1 of 1 START sql table model analytics.session_summary ........................ [RUN]
06:36:14  1 of 1 OK created sql table model analytics.session_summary ................... [SUCCESS 1
.31s]
06:36:14
06:36:14  Finished running 1 table model in 0 hours 0 minutes and 3.25 seconds (3.25s).
06:36:14
06:36:14  Completed successfully
06:36:14
06:36:14  Done. PASS=1 WARN=0 ERROR=0 SKIP=0 TOTAL=1
```

How the output table is built? (1)

```
WITH  __dbt__cte__user_session_channel as (
SELECT
    userId,
    sessionId,
    channel
FROM USER_DB_VIPER.raw.user_session_channel
),  __dbt__cte__session_timestamp as (
SELECT
    sessionId,
    ts
FROM USER_DB_VIPER.raw.session_timestamp
), u AS (
    SELECT * FROM __dbt__cte__user_session_channel
), st AS (
    SELECT * FROM __dbt__cte__session_timestamp
)
SELECT u.userId, u.sessionId, u.channel, st.ts
FROM u
JOIN st ON u.sessionId = st.sessionId
```

Content of target/compiled/build_mau/models/output/session_summary.sql

How the output table is built? (2)

```
create or replace transient table
USER_DB_VIPER.analytics.session_summary
as
      (WITH  __dbt__cte__user_session_channel as (
SELECT
   userId,
   sessionId,
   channel
FROM USER_DB_VIPER.raw.user_session_channel
WHERE sessionId IS NOT NULL
),  __dbt__cte__session_timestamp as (
SELECT
   sessionId,
   ts
FROM USER_DB_VIPER.raw.session_timestamp
WHERE sessionId IS NOT NULL
), u AS (
   SELECT * FROM __dbt__cte__user_session_channel
), st AS (
   SELECT * FROM __dbt__cte__session_timestamp
)
SELECT u.userId, u.sessionId, u.channel, st.ts
FROM u
JOIN st ON u.sessionId = st.sessionId
       );
```

Content of target/run/build_mau/models/output/session_summary.sql

Break

dbt Models - snapshot

What is a snapshot in databases?

- Dimension tables can frequently change depending on their nature
- In dbt, it refers to a feature allowing you to go back to any point in the past to view the table's contents
  - This enables rollback to past data if there's a problem with the table
  - It also makes debugging various data-related issues easier
  - Similar to time travel provided by Snowflake (a poor man’s **Time Travel**)

Types of Snapshots (SCD: Slowly Changing Dimension)

- SCD Type 0
  - Attributes that do not change. The original value is preserved.
- SCD Type 1
  - Updates data by overwriting the old value without keeping any history.
- **SCD Type 2**
  - Adds a new row for each change to track the change history
  - It often uses effective start and end date columns.
- SCD Type 3
  - Stores both the previous and current values in the same row
  - For example, it might keep both the current address and the previous address.

SCD Type 2 and dbt #1

- When data for a specific entity in the Dimension table changes
- Example: employee_jobs table
  - When the job_code for a specific employee_id changes
  - The time of change should also be added

EMPLOYEE_ID	JOB_CODE
E001	J01
E002	J02
E003	J02

J03

JOB CODE for E002 has been changed to J03

SCD Type 2 and dbt #2

- A separate table is created to maintain change history: snapshot table
- For a change, there will be a new record added to the table

JOB CODE for E002 has been changed to J03 from J02

EMPLOYEE_ID	JOB_CODE	**DBT_VALID_FROM**	**DBT_VALID_TO**
E002	J02	**2020-01-01**	**2023-02-01**
E002	**J03**	**2023-02-01**	**NULL**

In addition, two more fields are added: dbt_scd_id & dbt_updated_at

dbt's snapshot processing method

1. First, configuration is done in the snapshots folder
2. To perform snapshots, the data source must meet certain conditions:
   - A primary key must exist
   - A timestamp indicating when records were changed is needed (e.g., updated_at, modified_at)
3. Criteria for detecting changes:
   - Based on the primary key, when the change time is later than the time currently in the DW
4. The snapshots table contains a total of 4 timestamps:
   - dbt_scd_id, dbt_updated_at
   - dbt_valid_from, dbt_valid_to

snapshots - snapshot_session_summary.sql

```
{% snapshot snapshot_session_summary %}

{{
  config(
    target_schema='snapshot',
    unique_key='sessionId',
    strategy='timestamp',
    updated_at='ts',
    invalidate_hard_deletes=True
  )
}}

SELECT * FROM {{ ref('session_summary') }}

{% endsnapshot %}
```

Create **snapshot** schema in Snowflake or specify **analytics**

dbt snapshot

```
keeyong build_mau % dbt snapshot
06:54:21  Running with dbt=1.9.3
06:54:22  Registered adapter: snowflake=1.9.2
06:54:22  Found 3 models, 1 snapshot, 3 sources, 474 macros
06:54:22
06:54:22  Concurrency: 1 threads (target='dev')
06:54:22
06:54:23  1 of 1 START snapshot snapshot.snapshot_session_summary ........................ [RUN]
06:54:26  1 of 1 OK snapshotted snapshot.snapshot_session_summary ........................ [SUCCESS 1
.50s]
06:54:26
06:54:26  Finished running 1 snapshot in 0 hours 0 minutes and 3.69 seconds (3.69s).
06:54:26
06:54:26  Completed successfully
```

Check snapshot.snapshot_session_summary table

```
SELECT *
FROM snapshot.snapshot_session_summary
LIMIT 10;
```

USERID	SESSIONID	CHANNEL	TS	DBT_SCD_ID	DBT_UPDATED_AT	DBT_VALID_FROM	DBT_VALID_TO
779	7cdace91c487558e27ce54df7cdb299c	Instagram	2019-05-01 00:13:11.783	968a20fba603a2f1669abf0e39255815	2019-05-01 00:13:11.783	2019-05-01 00:13:11.783	null
230	94f192dee566b018e0acf31e1f99a2d9	Naver	2019-05-01 00:49:46.073	d1484d42374543038d705da14d6259ab	2019-05-01 00:49:46.073	2019-05-01 00:49:46.073	null
369	7ed2d3454c5eea71148b11d0c25104ff	Youtube	2019-05-01 10:18:43.210	8fdecd2c89c86d082fe65756e64c7b83	2019-05-01 10:18:43.210	2019-05-01 10:18:43.210	null
248	f1daf122cde863010844459363cd31db	Naver	2019-05-01 13:10:56.413	3140d8abcb37b1fcf782c3fd72685fe0	2019-05-01 13:10:56.413	2019-05-01 13:10:56.413	null
676	fd0efcca272f704a760c3b61dcc70fd0	Instagram	2019-05-01 13:45:19.793	5d6a0aab72232c7f67f4420e2a870bd7	2019-05-01 13:45:19.793	2019-05-01 13:45:19.793	null
40	8804f94e16ba5b680e239a554a08f7d2	Youtube	2019-05-01 14:23:07.660	c068e54e00def6d8edb661f6ce9e8035	2019-05-01 14:23:07.660	2019-05-01 14:23:07.660	null
468	c5f441cd5f43eb2f2c024e1f8b5d00cd	Instagram	2019-05-01 15:03:54.650	d73ce4e551a457d41304cbda7fc0b4c3	2019-05-01 15:03:54.650	2019-05-01 15:03:54.650	null
69	d5fcc35c94879a4afad61cacca56192c	Facebook	2019-05-01 15:13:16.140	1eaa183e4b8ebab807df02bbdbc93a3b	2019-05-01 15:13:16.140	2019-05-01 15:13:16.140	null
420	3d191ef6e236bd1b9bdb9ff4743c47fe	Youtube	2019-05-01 15:33:58.197	5536b40dc468a35cb851a422e8ae6a14	2019-05-01 15:33:58.197	2019-05-01 15:33:58.197	null
572	c17028c9b6e0c5deaad29665d582284a	Organic	2019-05-01 15:59:57.490	439009581c657cd1ac0345f80e8c5e90	2019-05-01 15:59:57.490	2019-05-01 15:59:57.490	null

Snapshot Practice Overall Steps

- From Snowflake Web UI, update a record’s channel field in session_summary
- Run “dbt snapshot”
- Check snapshot.snapshot_session_summary table

Snapshot practice #2

- From Snowflake Web UI, update a record's channel field in session_summary

```
SELECT *
FROM analytics.session_summary
WHERE sessionid = '7cdace91c487558e27ce54df7cdb299c';
```

USERID	SESSIONID	CHANNEL	TS
779	7cdace91c487558e27ce54df7cdb299c	Instagram	2019-05-01 00:13:11.783

```
UPDATE analytics.session_summary
SET channel = 'Google', ts = GETDATE()
WHERE sessionid = '7cdace91c487558e27ce54df7cdb299c';
```

USERID	SESSIONID	CHANNEL	TS
779	7cdace91c487558e27ce54df7cdb299c	Google	2024-10-21 10:09:40.396

Snapshot practice #3

- Run “dbt snapshot”

```
keeyong build_mau % dbt snapshot
06:58:34  Running with dbt=1.9.3
06:58:35  Registered adapter: snowflake=1.9.2
06:58:36  Found 3 models, 1 snapshot, 3 sources, 474 macros
06:58:36
06:58:36  Concurrency: 1 threads (target='dev')
06:58:36
06:58:37  1 of 1 START snapshot snapshot.snapshot_session_summary ........................ [RUN]
06:58:41  1 of 1 OK snapshotted snapshot.snapshot_session_summary ........................ [SUCCESS 2
.07s]
06:58:41
06:58:41  Finished running 1 snapshot in 0 hours 0 minutes and 5.08 seconds (5.08s).
06:58:41
06:58:41  Completed successfully
06:58:41
06:58:41  Done. PASS=1 WARN=0 ERROR=0 SKIP=0 TOTAL=1
```

Snapshot practice #4

- Check to see if the record change is captured in Snowflake

```
SELECT *
FROM snapshot.snapshot_session_summary
WHERE sessionid = '7cdace91c487558e27ce54df7cdb299c';
```

USERID	SESSIONID	CHANNEL	TS	DBT_SCD_ID	DBT_UPDATED_AT	DBT_VALID_FROM	DBT_VALID_TO
779	7cdace91c487558e27ce54df7cdb299c	Google	2024-10-21 10:09:40.396	b98bfb015f073bd5ca5377	2024-10-21 10:09:40.3	2024-10-21 10:09:40.3	null
779	7cdace91c487558e27ce54df7cdb299c	Instagram	2019-05-01 00:13:11.783	968a20fba603a2f1669abf	2019-05-01 00:13:11.7	2019-05-01 00:13:11.7	2024-10-21 10:09:

dbt test

How to Conduct Testing in dbt

- Methods for testing data quality
- Two types exist:
  - Built-in general tests ("Generic")
    - Supports tests such as unique, not_null, accepted_values, relationships, etc.
    - Located in the models folder
  - Custom tests ("Singular")
    - Basically simple SELECT statements where a returned result is considered a "failure"
    - Located in the tests folder

How to Conduct Generic Testing

- Implement in models/schema.yml
- Tests can be added per table per column
  - unique,
  - not_null,
  - accepted_values
  - relationship
  - dbt_expectation tests
    - expect_column_values_to_be_between, expect_column_values_to_match_regex, expect_column_values_to_be_in_set, expect_column_pair_values_to_be_equal, expect_column_sum_to_equal, …
  - dbt_utils tests
    - equality, recency, at_least_one, cardinality_equality, sequential_values
  - For a specific column of the table, we can do these testing

models - schema.yml

```yaml
version: 2

models:
  - name: session_summary
    description: "Analytics model for session data"
    columns:
      - name: sessionId
        description: "Unique identifier for each session"
        data_tests:
          - unique
          - not_null
```

dbt test

```
keeyong build_mau % dbt test
06:59:26  Running with dbt=1.8.7
06:59:26  Registered adapter: snowflake=1.8.3
06:59:27  Found 3 models, 1 snapshot, 2 data tests, 3 sources, 454 macros
06:59:27
06:59:29  Concurrency: 1 threads (target='dev')
06:59:29
06:59:29  1 of 2 START test not_null_session_summary_sessionId ..........................
. [RUN]
06:59:30  1 of 2 PASS not_null_session_summary_sessionId ...............................
. [PASS in 1.64s]
06:59:30  2 of 2 START test unique_session_summary_sessionId ............................
. [RUN]
06:59:31  2 of 2 PASS unique_session_summary_sessionId ..................................
. [PASS in 0.63s]
06:59:31
06:59:31  Finished running 2 data tests in 0 hours 0 minutes and 4.02 seconds (4.02s).
06:59:31
06:59:31  Completed successfully
06:59:31
06:59:31  Done. PASS=2 WARN=0 ERROR=0 SKIP=0 TOTAL=2
```

dbt docs

Introduction to dbt Documentation

- Philosophy is to keep documentation as close as possible to the SQL
- There are two ways to handle documentation
  - Adding documentation directly to existing .yml files (preferred method)
  - Creating standalone markdown files
- This documentation can be served using a lightweight web server
  - overview.md will act as the default homepage
  - It is also possible to include assets like images.

How to document?

- Add description to entries in models/schema.yml and models/sources.yml

```yaml
version: 2

models:
  - name: session_summary
    description: "Analytics model for session data"
    columns:
      - name: sessionId
        description: "Unique identifier for each stock price record"
        data_tests:
          - unique
          - not_null
```

dbt docs generate

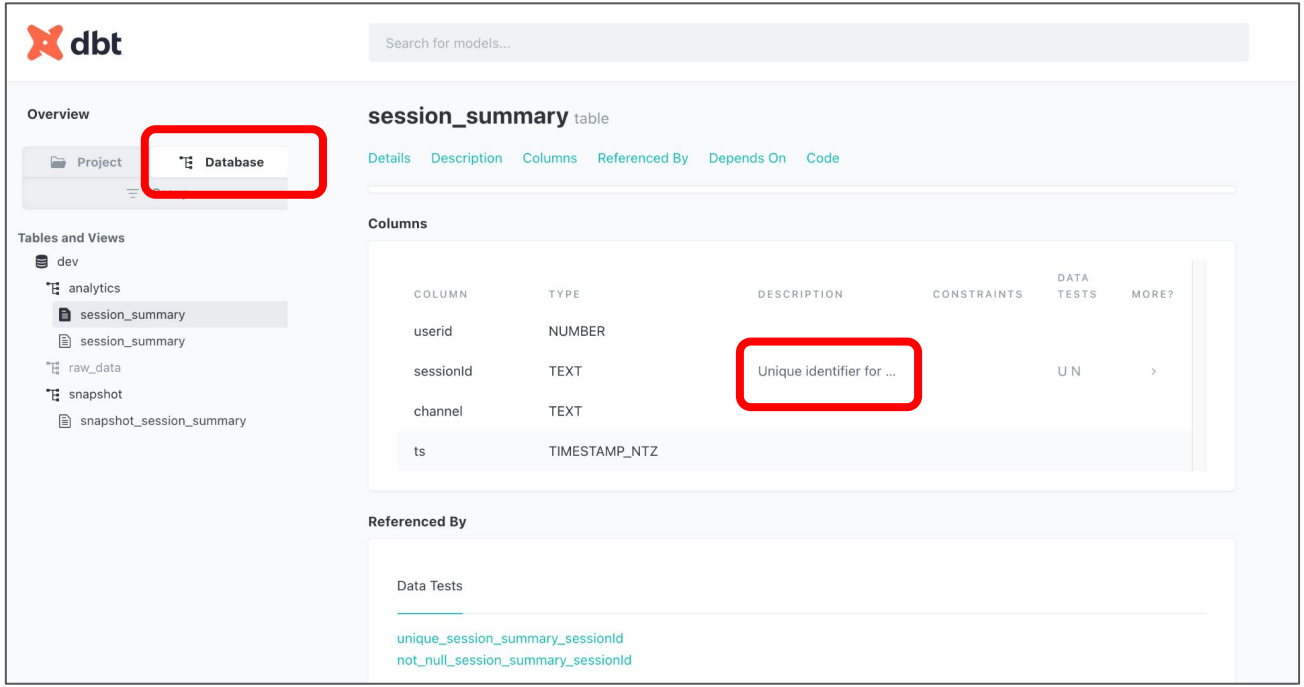
- If the user has higher permissions, more information can be retrieved from Snowflake and displayed
- The resulting file will be saved as target/catalog.json

```
keeyong build_mau % dbt docs generate
07:01:10  Running with dbt=1.8.7
07:01:10  Registered adapter: snowflake=1.8.3
07:01:11  Found 3 models, 1 snapshot, 2 data tests, 3 sources, 454 macros
07:01:11
07:01:12  Concurrency: 1 threads (target='dev')
07:01:12
07:01:12  Building catalog
07:01:17  Catalog written to /Users/jobox/Downloads/SJSU/sjsu-data226/week10/build_mau/ta
rget/catalog.json
```

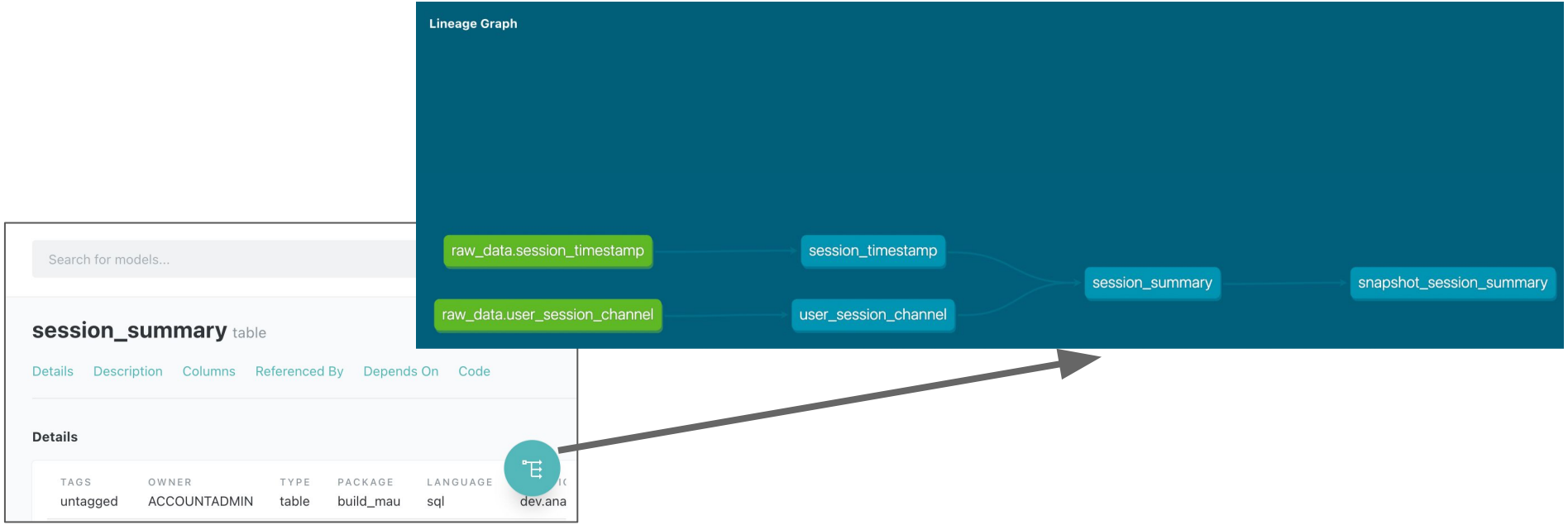
Check the Documentation as a web server

- dbt docs serve

dbt model
Physical table



Check the Lineage Graph



Integration of dbt & Airflow

Scheduling a dbt in Airflow

How to run dbt inside Airflow

The exact steps differ between Cloud Composer and Docker

1. Set up a dbt project folder
2. Mirror it with Airflow
3. Add dbt-snowflake python package to your Airflow setting
4. Build a DAG to schedule different dbt jobs

More details can be found at this doc

1. Set up dbt project folder

- Rename the project folder if needed. In my case, renamed to “dbt”
- You need profiles.yml file in the dbt root folder (check the content)
- The dbt project don’t need target folders so remove them by running “dbt clean”

```
keeyong sjsu-data226-SP25 % ls -tl week10
total 8
drwxr-xr-x  8 jobox  staff   256 Mar 27 00:18 dbt
-rw-r--r--  1 jobox  staff  1846 Mar 27 00:18 build_elt_with_dbt.py
keeyong sjsu-data226-SP25 % ls -tl week10/dbt
total 24
drwxr-xr-x  3 jobox  staff    96 Mar 27 00:18 snapshots
-rw-r--r--  1 jobox  staff   398 Mar 27 00:18 profiles.yml
```

```
keeyong dbt % dbt clean
18:05:11  Running with dbt=1.8.7
18:05:11  Checking /Users/jobox/Downloads/SJSU/sjsu-data226/week8/airflow/dbt/
18:05:11  Cleaned /Users/jobox/Downloads/SJSU/sjsu-data226/week8/airflow/dbt/t
18:05:11  Checking /Users/jobox/Downloads/SJSU/sjsu-data226/week8/airflow/dbt/
18:05:11  Cleaned /Users/jobox/Downloads/SJSU/sjsu-data226/week8/airflow/dbt/d
18:05:11  Finished cleaning all paths.
```

2. Mirror it with Airflow (#2)

- The folder should be at the same level as dags folder
  - Docker: This needs to be mirrored in the case of Docker Airflow - “docker-compose.yaml”

```
keeyong airflow % ls -tl
total 72
drwxr-xr-x  13 jobox  staff    416 Oct 15 23:19 dbt
drwxr-xr-x@  8 jobox  staff    256 Oct 15 23:15 dags
-rw-r--r--   1 jobox  staff   8683 Oct 15 21:13 docker-compose-min.yaml
-rw-r--r--   1 jobox  staff  11425 Oct 15 18:14 docker-compose.yaml
drwxr-xr-x@  9 jobox  staff    288 Oct 14 11:54 logs
drwxr-xr-x@  3 jobox  staff     96 Oct 12 14:09 plugins
drwxr-xr-x   3 jobox  staff     96 Oct 12 14:09 config
```

```
volumes:
  - ${AIRFLOW_PROJ_DIR:-.}/dags:/opt/airflow/dags
  - ${AIRFLOW_PROJ_DIR:-.}/logs:/opt/airflow/logs
  - ${AIRFLOW_PROJ_DIR:-.}/config:/opt/airflow/config
  - ${AIRFLOW_PROJ_DIR:-.}/plugins:/opt/airflow/plugins
  - ${AIRFLOW_PROJ_DIR:-.}/dbt:/opt/airflow/dbt
```

3. Add dbt-snowflake python package to your Airflow

- Docker: Add it to _PIP_ADDITIONAL_REQUIREMENTS in docker-compose file

docker-compose.yaml

```
…
# for other purpose (development, test and especially production usage) build/extend Airflow image.
_PIP_ADDITIONAL_REQUIREMENTS: ${_PIP_ADDITIONAL_REQUIREMENTS:- yfinance
apache-airflow-providers-snowflake snowflake-connector-python dbt-snowflake}
# The following line can be used to set a custom config file, stored in the local config folder
…
```

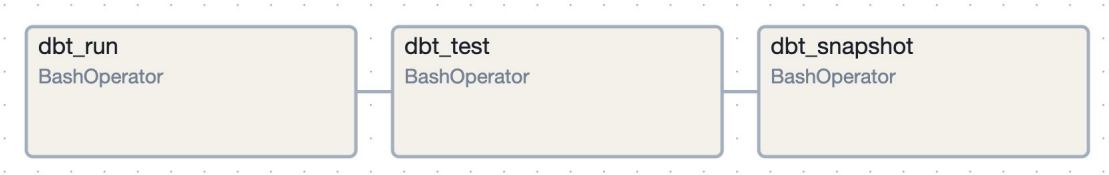
Make sure to restart your Docker Containers:

- Stop -> Start
- docker compose down -> docker compose up

4. Build a DAG to schedule different dbt jobs

- Essentially running different dbt commands via BashOperator
  - BashOperator is to run linux commands
- Source Code
  - How to pass the Snowflake credentials to Environment Variables
  - Make sure you set up your Snowflake connections properly
  - Copy the file to dags folder: cp week10/build_elt_with_dbt.py dags/

```
default_args={
    "env": {
        "DBT_USER": "{{ conn.snowflake_conn.login }}",
        "DBT_PASSWORD": "{{ conn.snowflake_conn.password }}",
        "DBT_ACCOUNT": "{{ conn.snowflake_conn.extra_dejson.account }}",
        ...
    }
}
```



Lab #2 Details

Lab #2 Overview (50 pts)

- Due date: 11/25
- Lab team: if you want to change, talk to me
- The goal is to build an ELT system with a dashboard
  - The dashboard of your choice can be one of Preset, Superset Docker, Tableau or any tool
  - What datasets to use
    i. This can be an extension of what you built in Lab #1
    ii. This can be something new focusing more on any historical data and insight from it

Lab #2 Requirements

1. Populate raw data tables using Airflow
    a. Reuse your work from Lab 1 if applicable
    b. For new datasets, implement an ETL job in Airflow
    c. Capture screenshots accordingly
2. Create abstract table(s) using dbt
    a. Schedule dbt job(s) from Airflow
    b. Capture screenshots accordingly
3. Visualize insights using Preset or Superset or Tableau
    a. Base visualizations on the table(s) created in step 2
    b. Capture screenshots accordingly
4. Prepare the report in IEEE format
    a. Single-column format is acceptable, provided that it includes the same sections

All codes (including ETLs and dbt ELT) need to be in Github repository

Lab #2 Idea if you stick to the stock price project

1. Transformations in dbt
   a. Perform specific informative transformations (e.g., Moving Averages, RSI) in dbt.
   b. Once transformations are built, ensure the dbt models are run as part of an Airflow DAG.
2. Visualize Data in a BI tool (Superset or Preset or Tableau)
   a. Use a BI tool to visualize key metrics such as moving averages, RSI, price momentum, and so on you built from the step 1

Homework #7

Homework #7 (9 points)

Follow today's class instructions (dbt only. No need to run it as a DAG)

1. Create a dbt project with snowflake connector (1 pt)
2. Set up Input models (2pt)
   a. Input tables should be built as CTE
   b. Show contents of models/input/user_session_channel.sql and input/session_timestamp.sql
3. Set up Output models (1pt)
   a. Show contents of models/output/session_summary.sql
4. Add Snapshot to the output table (session_summary) (2pt)
   a. Show contents of snapshots/snapshot_session_summary.sql
5. Add at least 2 tests to sessionId field of session_summary (2pt)
   a. Show contents of models/schema.yml

- For each step, capture screenshots accordingly (examples to follow)
- Create a github repo for this and add the link to the report

Screenshot from step 1 (Show your dbt init command)

```
Which database would you like to use?
[1] snowflake

(Don't see the one you want? https://docs.getdbt.com/docs/available-adapters)

Enter a number: 1
account (https://<this_value>.snowflakecomputing.com): icduxid-fzb33116
user (dev username): keeyonghan
[1] password
[2] keypair
[3] sso
Desired authentication type option (enter a number): 1
password (dev password):
role (dev role): ACCOUNTADMIN
warehouse (warehouse name): compute_wh
database (default database that dbt will build objects in): dev
schema (default schema that dbt will build objects in):
schema (default schema that dbt will build objects in): analytics
threads (1 or more) [1]: 1
21:58:18  Profile stock_price_analysis written to /Users/jobox/.dbt/profiles.yml
using target's profile_template.yml and your supplied values. Run 'dbt debug' to
validate the connection.
```

Screenshot from step 2 (dbt run from slide #36)

```
keeyong build_mau % dbt run
06:21:10  Running with dbt=1.8.7
06:21:11  Registered adapter: snowflake=1.8.3
06:21:11  Unable to do partial parsing because a project config has changed
06:21:11  [WARNING]: Configuration paths exist in your dbt_project.yml file which
 do not apply to any resources.
There are 1 unused configuration paths:
- models.build_mau.output
06:21:12  Found 2 models, 3 sources, 454 macros
06:21:12
06:21:12  Concurrency: 1 threads (target='dev')
06:21:12
06:21:12
06:21:12  Finished running  in 0 hours 0 minutes and 0.02 seconds (0.02s).
```

Screenshot from step 3 (dbt run from slide #40)

```
keeyong build_mau % dbt run
06:34:41  Running with dbt=1.8.7
06:34:41  Registered adapter: snowflake=1.8.3
06:34:41  Found 3 models, 3 sources, 454 macros
06:34:41
06:34:44  Concurrency: 1 threads (target='dev')
06:34:44
06:34:44  1 of 1 START sql table model analytics.session_summary ................
......... [RUN]
06:34:46  1 of 1 OK created sql table model analytics.session_summary ...........
......... [SUCCESS 1 in 2.60s]
06:34:46
06:34:46  Finished running 1 table model in 0 hours 0 minutes and 5.07 seconds (5
.07s).
06:34:46
06:34:46  Completed successfully
06:34:46
06:34:46  Done. PASS=1 WARN=0 ERROR=0 SKIP=0 TOTAL=1
```

Screenshot from step 4 (dbt snapshot from slide 54)

```
keeyong build_mau % dbt snapshot
06:52:32  Running with dbt=1.8.7
06:52:33  Registered adapter: snowflake=1.8.3
06:52:33  Found 3 models, 1 snapshot, 3 sources, 454 macros
06:52:33
06:52:35  Concurrency: 1 threads (target='dev')
06:52:35
06:52:35  1 of 1 START snapshot snapshot.snapshot_session_summary ...............
......... [RUN]
06:52:38  1 of 1 OK snapshotted snapshot.snapshot_session_summary ...............
......... [SUCCESS 1 in 2.84s]
06:52:38
06:52:38  Finished running 1 snapshot in 0 hours 0 minutes and 5.16 seconds (5.16
s).
06:52:38
06:52:38  Completed successfully
06:52:38
06:52:38  Done. PASS=1 WARN=0 ERROR=0 SKIP=0 TOTAL=1
```

Screenshot from step 5 (dbt test from slide 64)

```
keeyong build_mau % dbt test
06:59:26  Running with dbt=1.8.7
06:59:26  Registered adapter: snowflake=1.8.3
06:59:27  Found 3 models, 1 snapshot, 2 data tests, 3 sources, 454 macros
06:59:27
06:59:29  Concurrency: 1 threads (target='dev')
06:59:29
06:59:29  1 of 2 START test not_null_session_summary_sessionId ..........................
. [RUN]
06:59:30  1 of 2 PASS not_null_session_summary_sessionId ...............................
. [PASS in 1.64s]
06:59:30  2 of 2 START test unique_session_summary_sessionId ............................
. [RUN]
06:59:31  2 of 2 PASS unique_session_summary_sessionId ................................
. [PASS in 0.63s]
06:59:31
06:59:31  Finished running 2 data tests in 0 hours 0 minutes and 4.02 seconds (4.02s).
06:59:31
06:59:31  Completed successfully
06:59:31
06:59:31  Done. PASS=2 WARN=0 ERROR=0 SKIP=0 TOTAL=2
```

Week 11 (NoSQL - VectorDB)